高度情報演習2C

テーマ1：

「インタラクティブなアプリケーションの設計と実装および評価」

真鍋担当分

テーマ1（1/6）
ポインティングタスク

この演習で行うこと

- Unityを使って○○を作る，実験する，レポート書く
- 0回目：**環境セットアップの宿題**
- 1回目：ルーレットアプリ，ポインティングタスクの実装
- 2回目：ポインティングタスクの実装評価，**論文宿題**
- 3回目：ポインティングタスクの評価，**レポート**
- 4回目：2種類の入力手法の決定と実装
- 5回目：実装した入力手法の評価
- 6回目：成果発表会，**レポート**

	1回目	2回目	3回目	4回目	5回目	6回目
前半：	10/07,	10/14,	10/21,	10/28,	11/11,	11/18
後半：	11/25,	12/02,	12/09,	12/16,	12/23,	01/13

ある人たちの会話

- Aさん：「やっぱり○○のマウスは最高だよね」
- Bさん：「えっ，トラックポイント一択じゃないの？」
- Cさん：「私はトラックボール派だけど」
- Dさん：「板タブってマイナーなのかな．．．」

あなたは，何をする？
あなたは，何ができる？

ポインティングデバイス

- マウス，タッチ，トラックボール，トラックポイント，，，
- それらは何をするもの？
- 実物を触ってみよう

- **どれが良いのだろうか？，なぜ良いのだろうか？**
  - ユーザはどのように慣れていくのか？
  - AさんとBさんはどちらの方が上手なのか？
  - より良いものとはどんなものなのか？を知りたい
  - ポインティングをモデル化したい

ポインティングタスク

- **GUIにおける操作の流れ**
  - 何かを選択する，と決める
  - クリックする対象を決める
  - カーソルを移動させる
  - 対象の上にカーソルが移動したらクリックする
  - 対象が選択されたことを理解する
  - 選択した結果が正しいことを理解する

定性的なモデル

- **ポインティングタスク**
  - ポインティングの操作だけに注目
    - 選択することは前提．対象が明らか．選択結果が正しいかどうかを知る必要がない
  - ポインティングデバイスの優劣を比較することができる

定量化して比較したい

- **結果を定量化させたい**
  - 定量：数字で表す，定性：形容詞で表す
  - 定性評価の例：
    - Aさんの方が優れている．デバイスAの方が操作しづらい
    - 忙しい時にはBの方が仕事が早い
  - 定量評価の例：
    - Aさんの効率は2倍．デバイスAの誤入力は3倍
    - タスクの数が10あるとき，全タスク達成時間はBの方が20%短い

- **ポインティングタスクの定量化**
  - どうすれば，何をすれば，定量化できる？
  - ＝ポインティングタスクの定量的なモデル化

例えば　その1．モデル化

- ターゲットまでの距離
- ターゲットの大きさ
- ターゲットの形，色
- ターゲットの存在する方向
- 時間
- 移動距離
- カーソル速度
- 正解率，失敗率
- カーソルの大きさ
- カーソルの形，色，など

ターゲットまでの距離が長くなると，何がどうなる？

ターゲットの大きさが大きくなると，何がどうなる？

カーソルの大きさが大きくなると，何がどうなる？

複数のパラメータの関係を調べる

例えば　その2．比較

- ターゲットまでの距離
- ターゲットの大きさ
- ターゲットの形，色
- ターゲットの存在する方向
- 時間
- 移動距離
- カーソル速度
- 正解率，失敗率
- カーソルの大きさ
- カーソルの形，色，など

何をパラメータとすれば2つの手法やデバイス，ユーザを比較できる？

複数の手法を比較するのに適切なパラメータを選定する

とある人たちの会話　その2

- Aさん：「ターゲットまで遠いとポインティングに時間がかかるのは当たり前じゃん」
- Bさん：「遠いとカーソル速度が上がるから，トータルの時間なんて変わらないよ」
- Cさん：「いや，1ピクセルだけ動かす時って，すごく慎重になるから，むしろ近い方が遅い気がする」

あなたは，何をする？
あなたは，何ができる？

今回やること

- **Step1**
  - **「ターゲットまでの距離が遠い場合，ポインティングが完了するまでの時間は長い」を検証する**
    - 遠い場合と近い場合の比較

最低限，ここまでは必須

- **Step2**
  - **ターゲットまでの距離とタスク完了時間の関係**を明らかにする
    - ポインティングタスクのモデル化

ここまで，できてほしい

- **Step3**
  - **距離＋別なパラメータと，タスク完了時間の関係**を明らかに
  - ポインティングタスクにおける**複数デバイスの比較**

ここまでできたら，すばらしい

Unityでポインティングタスクを実装

- できあがりイメージはこんな感じ
  - リンク
  - リンク



- 必要な要素
  - ターゲット
  - カーソル
  - 結果表示，ファイル出力
- 動作
  - カーソル位置はマウスに連動
  - クリックしたらターゲットの位置が変化
  - ターゲットを正しく選択できているかを判定
  - ターゲット表示からクリックまでの時間を計測

どのように実装する？

- **方法その1**
  - マウスカーソルは標準のまま
  - マウスクリック時に，その下のオブジェクトを特定
  - クリックしたオブジェクトがターゲットかどうかを判定
- **方法その2**
  - 標準カーソルを消し，マウスに連動するオブジェクトを作成
  - マウスクリック時に，マウス連動オブジェクトとターゲットが衝突しているかを判定

- 方法1の方が楽だけど，自由度が低くなってしまう
- 方法2は，自由度が増すけれども実装が問題になる
- まずは，方法1で実装してみましょう

どのオブジェクトに何を実装する？

- **どこでやる？どこに実装する？何が必要？**
  - クリックを検出
  - クリックの成功・失敗を判断
  - ターゲット位置の変更
  - 時間の管理
  - 画面への表示

スタートと終了はどうする？

- いきなりスタートさせる？
- どうなれば終了する？どうやって終了させる？

データ

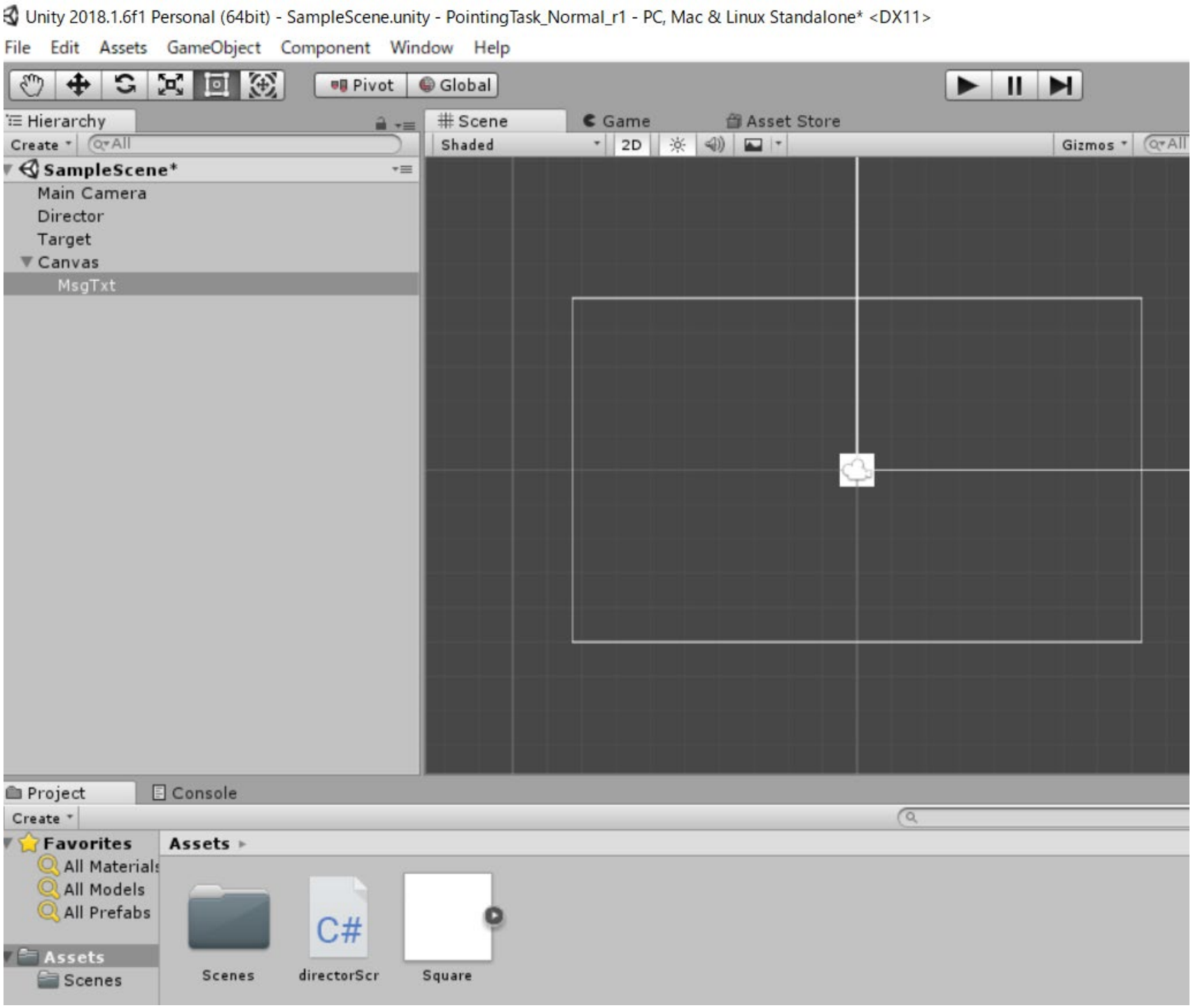
- どんなデータが必要だろうか？
- どうやってデータを取り出すか？どうやって書き出すか？

- 何をしたいかに強く依存する
  - はじめから全てのデータを取得しておく
  - やりながら何をしたいのかを考えていく

- **ここはかなり真剣に考えるところ！**

作る上でのTips 1

- Hierarchy windowには,
  - Main Camera
  - Director
    - EmptyなGameObject．ハイアラキーウィンドウ→Create Empty
    - ゲーム全体を管理
  - Target
    - スプライト．Project→Create→Spritesから作る
    - クリックされるターゲット．**Collider 2Dコンポーネント**をつける.
  - Canvas
    - UIのText．**ハイアラキーウィンドウ→UI→Text**で生成
    - 結果表示用．まずはPositionを0,0,0にしておく
- Project windowには,
  - director
    - C#スクリプト．Directorにアタッチする
  - Squareなどスプライト
    - これをHierarchyに持っていってTargetにする


Unity 2018.1.6f1 Personal (64bit) - SampleScene.unity - PointingTask_Normal_r1 - PC, Mac & Linux Standalone* <DX11>
File Edit Assets GameObject Component Window Help
Pivot Global
Hierarchy
Create All
SampleScene*
Main Camera
Director
Target
Canvas
MsgTxt
Scene Game Asset Store
Shaded 2D Gizmos All
Project Console
Create
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Assets
Scenes
directorScr
Square

作る上でのTips 2

- C#スクリプト
  - クリックイベント発生時に処理を行う

```
void Update() {
    if (Input.GetMouseButtonDown(0))
    {
       //    クリックされた時の処理を書く
    }
}
```

作る上でのTips 3

- C#スクリプト
  - 表示内容を変える
  - ハイアラキーウィンドウのTextはMsgTxtにリネームされたと仮定

青文字：TextMeshProを使う場合，**赤文字：UITextを使う場合**，黒文字：共通

```
using UnityEngine.UI;
Using TMPro;

public class directorScr : MonoBehaviour {
    GameObject msgtxt;

    // Use this for initialization
    void Start () {
            msgtxt = GameObject.Find("MsgTxt");
            msgtxt.GetComponent<Text>().text = "click to start";
            msgtxt.GetComponent<TextMeshProGUI>.text=“click to start”;
     }
```

作る上でのTips 4

- クリックされたら，その下のオブジェクト（のcollider）を取得する

```
if (Input.GetMouseButtonDown(0))
{
    Vector2 clickPoint = Camera.main.ScreenToWorldPoint(Input.mousePosition);
    Collider2D collision2d = Physics2D.OverlapPoint(clickPoint);
     if (collision2d)
     {
       //     クリックされた GameObject clickedObjectを取得
         clickedObject = collision2d.transform.gameObject;
     }
     else
     {
       //     その下にオブジェクトがない
     }
}
```

作る上でのTips 5

- targetの位置と大きさをランダムに変える

```
target.transform.position =
new Vector2(Random.Range(-7.0f, 7.0f), Random.Range(-4.0f, 4.0f));

target.transform.localScale = new Vector2(scale, scale);
```

作る上でのTips 6

- コンソールにデバッグ用のテキスト出力

```
Debug.Log(“これでConsoleにテキスト出力ができるよ”);
```

- 経過時間取得

```
float start_time, diff_time;

start_time = Time.time;
// 何かの処理
diff_time = Time.time – start_time;
```

作る上でのTips 7

- ファイルへの書き出し
  - データを取得した都度，追記していく場合のコード例

```
void WritePointingData( string txt)
{
    using (StreamWriter stream_writer = new StreamWriter("./PointingLog.txt", true))
    {
        stream_writer.WriteLine( text);
        stream_writer.Close();
    }
}
```

作る上でのTips 8

- 現在時刻を取得する
  - DateTimeを使う
  - 文字列にするときには，例えばToLongTimeString()

```
using System;

（省略）

DateTime.Now
```

```
DateTime.Now.ToLongTimeString()
```

作る上でのTips 9

- アプリケーションを終了させる
  - escを押したときに終了させる例
  - エディタの場合とスタンドアローンの場合の両対応

```
void Quit()
{
    #if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
    #elif UNITY_STANDALONE
        UnityEngine.Application.Quit();
    #endif
}

void Update ()
{
    if (Input.GetKey(KeyCode.Escape)) Quit();
}
```

作る上でのTips 10

- 標準のマウスカーソルを移動させる
  - win32APIを使う

```
using System.Runtime.InteropServices;

public class Director1 : MonoBehaviour
{
    [DllImport("user32.dll")]
    public static extern bool SetCursorPos(int x, int y);

    SetCursorPos(1024, 500);
}
```

来週までの宿題

- 来週ポインティングタスクの実験ができるよう，アプリを作っておいてください
  - 各自1つ．条件，設定は各自が考える
  - **他の学生のコピペはしない**
    - 難しいのであればタスクを簡単にする＝アイデアで勝負せよ
    - 去年の先輩のコピーもNG
  - 他人に使ってもらえるよう，自分でリハーサルしておく
    - 初めての人でも使えるか．
    - どのような指示を出せばよいのか．
    - どれくらい時間がかかるのか
  - 細かな数値を簡単に変えられるよう，パラメータは変数で持っておく（コードに数字を直接書き込まない）

宿題2

論文を読んでくる（予告）

宿題　リスト1

- 以下の9本の論文，あるいは自分で探した論文の中から2本以上読んでくること
  - 自分で選ぶ場合，8ページ以上であることが必須条件

1. Ninja cursors: using multiple cursors to assist target acquisition on large screens
   - https://doi.org/10.1145/1357054.1357201
2. 複数ダミーカーソル中における自分自身のカーソル特定
   - http://www.interaction-ipsj.org/archives/paper2013/data/Interaction2013/oral/data/pdf/13INT004.pdf
3. GUI環境でのバブルカーソルの実用的評価
   - https://ipsj.ixsq.nii.ac.jp/records/9782
4. ポインティングタスク中のピーク速度を用いたターゲット予測インタフェースの提案
   - https://ipsj.ixsq.nii.ac.jp/records/10075

5. 重なりあったウィンドウ間を移動可能なマウスカーソル操作手法とその評価
   - https://www.wiss.org/WISS2011Proceedings/PDF/12.pdf
6. Drag&Flick：タッチスクリーンを用いた 視覚障がい者向け文字入力方式
   - http://www.interaction-ipsj.org/archives/paper2013/data/Interaction2013/oral/data/pdf/13INT010.pdf
7. 予測型テキスト入力システム POBox
   - http://www.pitecan.com/UnixMagazine/PDF/if9804.pdf
8. Flickey：超小型タッチスクリーン端末におけるフリックに基づくQWERTYキーボード
   - http://www.interaction-ipsj.org/proceedings/2016/data/pdf/16INT010.pdf
9. Valve Icon: オーバーシュート後に生成される壁を用いたポインティング高速化手法の提案
   - https://ipsj.ixsq.nii.ac.jp/records/200946

宿題の目的

- これから，2つ以上の入力手法を自分で選び，それらの実装・評価をしてもらいます．
  - **従来手法（通常手法，既存手法）と提案手法の2つ**
- 通常の手法にはどんな問題があり，提案手法はその問題をどのように解決しているのかを理解してください
- また，提案手法が有効であることを，どのようにして主張しているのかを理解してください

- **一般的な論文の流れ**
  - 既存手法には課題がある → その課題を解決する提案手法を考えた → 実装した → 既存手法と比較した → 結果，提案手法は有効

もっと知りたい人向け

- 論文を検索する時は，google scholarを使います
  - https://scholar.google.co.jp/



・全ての論文が無料で見られる訳ではありません
・SRAS経由ならば無料で見られる論文があります
・引用元の数字が大きい方がインパクトの強い論文
・英語で検索する方がよい
・各論文の最後に引用文献（参考文献）欄があるので，そこから辿るのも手

興味のありそうな論文を探す → その論文が引用している論文，その論文を引用している論文を探す，とやると効率的

もっと知りたい人向け2

- GeminiやChatGPTの **Deep research** を使う

ポインティング完了時間を短縮するための手法についてまとめてください．極力ACMかIEEEが出版する論文を引用するようにしてください．



マウスを用いたポインティングタスクにおける完了時間短縮手法の調査

はじめに

マウスで画面上のターゲットを選択するポインティングタスクは、デスクトップや大型ディスプレイなど様々な環境で日常的に行われている。このタスクの完成時間（target acquisition time）は、手の動き（運動空間）とポインタの移動量（表示空間）との関係を示すFittsの法則でモデル化される。Fittsの法則によれば、所定の幅のターゲットに対して距離が長いほど選択に時間がかかる。研究者たちはこの制約を克服し、ユーザの負担を軽減するために様々な改良手法を提案してきた。本報告では、ACMおよびIEEEで発表された主な論文を中心に、Fittsの法則に基づく改良、制御表示ゲインの動的調整、ターゲット拡張、選択支援などの技術と評価結果を体系的にまとめる。

Fittsの法則に基づく改善

ターゲットの拡張とタイミング

- Expanding Widgets（McGuffin & Balakrishnan, 2005） – ターゲットが動的に拡大すればモータ空間上の幅が広がり、Fittsの法則における有効幅が増える。実験ではターゲットを90%の距離移動後に拡大しても、常に静的ターゲットより高速に選択でき、最終的な拡大サイズが性能の主要因であることが示された dgp.toronto.edu 。拡大後のサイズを用いたFittsの法則で選択時間を概ね予測できる dgp.toronto.edu 。複数ターゲットが同時に拡大すると衝突やオクルージョンが発生するため、拡大のタイミングや方向の調整が必要になる。
- Beating Fitts' Law における拡張ターゲットまとめ（Balakrishnan, 2004） – レビュー論文では、拡張ターゲットにより得られる利点として、表示空間内のターゲットを動的に拡大することで運動空間でのターゲット幅を大きくし、選択時間を短縮することを説明している。遅いタイミングでも効果があり、最終サイズが重要であること、複数ターゲット時の設計

Deep researchはすごい
個々の論文を探すだけでなく，それらをまとめてくれる．

Deep researchは回数制限があります．利用は計画的に．

←の共有リンクは[ここ]

まず，Deep researchで大雑把な検索をかけて，その中から興味のある論文を読み，さらにその論文が引用しているものや，その論文を引用している論文を探してみる．あるいはさらにDeep researchをかける